

# Training and Evaluating Your First Model

Dr. Naila Marir · College of Engineering, Effat University

In Lab 1 you framed a problem and got to know a dataset. Today you turn it into a model — and, more importantly, into a number you could defend to a clinician.

**The goal of this lab:** take the data you explored in Lab 1 through stages 4, 5 and 6 — prepare it, train a model on it, and measure it honestly.

**Prerequisites** Lab 1 completed. You should be able to say what *recall* is and why we chose it.

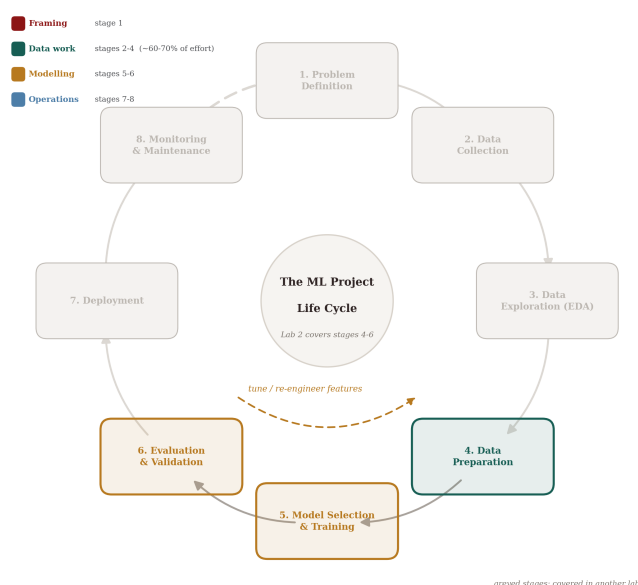
**Deliverable** The notebook with both # TODO cells completed, renamed MEAI502\_Lab2\_<YourName>.ipynb and uploaded to Blackboard.

**Assessment** Exercise 60 %, reflection 40 %. See §5.

**Notebook** MEAI502\_Lab2\_Student.ipynb — self-contained; it reloads the data itself.

## 1 Where We Left Off

≈ 10 minutes



**Figure 1 — Where you are.** Today covers stages 4 to 6. Stages 1 to 3 were Lab 1; 7 and 8 come later in the programme.

**Stage 1 (Lab 1).** Flag biopsies likely to be malignant so a radiologist reviews them first. Binary classification.  $y$ : 1 = malignant, 0 = benign. **Metric: recall on the malignant class** — of all the truly malignant cases, what fraction do we catch?

**Stages 2 and 3 (Lab 1).** 569 biopsies, 30 numeric features, no missing values, 37.3 % malignant, several features that clearly separate the two classes — and **wildly different scales** between columns. That last one becomes today's problem.

```
data = load_breast_cancer(as_frame=True)
X = data.data
y = (data.target == 0).astype(int)      # 1 = malignant, 0 = benign
```

## 2 Stage 4: Data Preparation

≈ 35 minutes

Stage 4 has two jobs today, and both matter more than which model you eventually pick: **split** the data, so you keep an honest test set; and **scale** the features, so the model can use them — without letting the test set influence training. Get either wrong and every number you report afterwards is fiction.

### 2.1 Split first

You need data the model has never seen, to estimate how it will behave on patients it has never seen. So hold some out now, before you touch anything else. Two arguments do real work:

- **stratify=y** keeps the malignant rate the same in both halves. Without it, a random split can hand you a test set with a different mix than the training set — and then you are measuring the wrong population.
- **random\_state=42** makes the split repeatable. Any fixed number works; the point is that you and I get the same rows.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=RANDOM_STATE)
```

*output*

```
malignant rate, whole dataset: 37.3%
malignant rate, training set : 37.3%
malignant rate, test set      : 37.1%
```

#### ⚠ THE TEST SET IS SEALED

We do not look at it, fit anything on it, or make any decision based on it until §4. Every time you peek at the test set and then change something, a little knowledge leaks into your choices and your final number becomes optimistic.

### 2.2 The scale problem

Remember what Lab 1 noticed: these columns live on completely different scales. `mean area` has a standard deviation in the hundreds; `mean concavity` has one below 0.1 — a difference of roughly four orders of magnitude.

Logistic regression learns one weight per feature and adds them up. If one feature arrives a thousand times larger than another, it dominates that sum for a reason that has nothing to do with the biology. So we put every feature on the same footing: **subtract the mean, divide by the standard deviation**. That is what `StandardScaler` does.

## 2.3 Scale — but fit on the training set only

This is the part to get exactly right.

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)    # LEARN from train, then apply
X_test_scaled  = scaler.transform(X_test)        # APPLY the same numbers to test
```

- **fit\_transform(X\_train)** — *learn* each column's mean and standard deviation from the training rows, then apply them.
- **transform(X\_test)** — apply **those same numbers** to the test rows. Do not re-learn them.

### ⚠ DATA LEAKAGE

If you learned the mean from the whole dataset, that mean would contain the test rows. Information from the test set would reach the training data, and your reported score would go *up* while your model got *worse*. It does not crash and it does not warn you, which is exactly why it is the most common bug in student projects.

Look at what changed. Every *training* column now has mean 0 and standard deviation 1. The *test* set does not:

```
output
TEST set, after scaling with the TRAINING set's numbers:
      mean area  mean concavity  mean texture
mean    -0.057         0.002      -0.102
std      0.894         0.937       1.016
```

That is not a bug — it is the whole point. The test set was transformed using numbers learned somewhere else, which is precisely the situation a deployed model faces: a new patient arrives and you scale them with the numbers you learned months ago in training. **If the test means had come out at exactly 0, it would mean you had leaked.**

---

## 3 Stage 5: Training

≈ 25 minutes

### 3.1 The baseline comes first

Before any real model: what does a *trivial* model score? `DummyClassifier(strategy="most_frequent")` always predicts the majority class — here, always "benign". This is the number your real model has to beat. Without it, "96 % accurate" means nothing at all.

*output*

```
BASELINE - always predicts benign
accuracy: 0.629
recall   : 0.000
```

There it is: **63 % accurate, and it catches nothing**. Zero recall. Every malignant case in the test set walks straight past it. If you only ever reported accuracy, this model would look like a reasonable start. That is the trap Lab 1 warned you about, made concrete.

### 3.2 The real model

`LogisticRegression` is a linear classifier: it learns one weight per feature, adds them up, and turns the total into a probability between 0 and 1. It is simple, fast, and — importantly for a medical tool — you can look at the weights and see what it is doing.

```
model = LogisticRegression(max_iter=5000, random_state=RANDOM_STATE)
model.fit(X_train_scaled, y_train)

y_pred = model.predict(X_test_scaled)
```

Two lines. This is the part everyone thinks machine learning *is*. Now the real question of this lab: **how do we say how good that is, in a way a clinician would accept?**

---

## 4 Stage 6: Evaluation

≈ 35 minutes

### 4.1 Start with accuracy, then stop trusting it

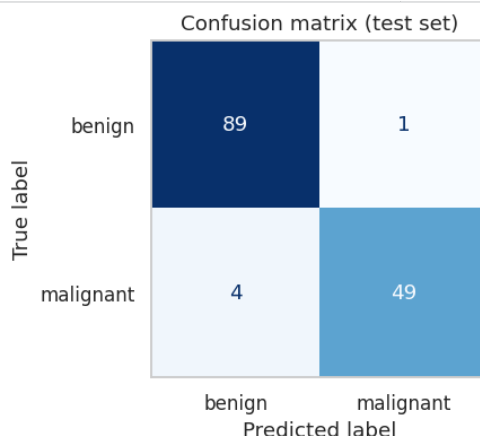
*output*

```
accuracy: 0.965
```

96.5 %, against a baseline of 62.9 %. Genuinely good — the model has learned something real. But accuracy tells you *how many* mistakes, never *which*. Our two mistakes are not equal: one costs a review, the other costs a life. So accuracy cannot be the last word.

### 4.2 The confusion matrix

The confusion matrix breaks the errors apart. Four numbers, and they are the foundation of every classification metric you will ever use.



**Figure 2 — Confusion matrix on the sealed test set.**

*output*

```

true negatives (TN) 89  benign, called benign
false positives (FP) 1  benign, wrongly flagged -> one extra review
false negatives (FN) 4  MALIGNANT, missed -> the expensive error
true positives (TP) 49  malignant, caught

```

Sit with those four numbers for a moment. **Four malignant cases were missed.** That is what "96.5 % accuracy" was hiding.

### 4.3 Recall and precision, by hand

Both come straight out of the four numbers above. Compute them yourself once — after that you can use the library function and know what it means.

| Metric           | Formula          | The question it answers                                      |
|------------------|------------------|--------------------------------------------------------------|
| <b>Recall</b>    | $TP / (TP + FN)$ | Of all the truly malignant cases, how many did we catch?     |
| <b>Precision</b> | $TP / (TP + FP)$ | Of all the cases we flagged, how many really were malignant? |

*output*

```

recall    = 49 / (49 + 4) = 0.925
precision = 49 / (49 + 1) = 0.980

```

### 4.4 The full report

`classification_report` gives you both classes at once. Read the **malignant** row.

*output*

|           | precision | recall | f1-score | support |
|-----------|-----------|--------|----------|---------|
| benign    | 0.96      | 0.99   | 0.97     | 90      |
| malignant | 0.98      | 0.92   | 0.95     | 53      |
| accuracy  |           |        | 0.97     | 143     |

**VERDICT****How you would say it out loud****TO THE PATHOLOGY LAB**

*On 143 biopsies it had never seen, the tool caught **92.5 %** of the malignant cases. Of the cases it flagged, **98 %** really were malignant, so it creates very few unnecessary re-views. It missed **4** malignant cases out of 53.*

That is a defensible statement: a metric that matches the decision, a number, and an honest account of what it costs. Compare it to "96.5 % accurate", which is true, higher, and tells the lab nothing they need.

**Is 92.5 % good enough?** That is not a question you answer alone — it is a conversation with the clinicians about what a miss costs. There are also ways to trade precision for recall, and better ways to check whether this number is stable. Both come in later labs.

---

**STAGES 7 & 8****What happens next** (no code)*≈ 10 minutes*

You have a model that works on a sealed test set. Two stages stand between that and something a hospital can use.

**STAGE 7****Deployment**

The model gets saved and served where the decision is made. One rule matters more than the rest: **the scaler has to travel with the model**. Your model was trained on scaled numbers, so production must scale incoming patients with *exactly* the numbers in `scaler` — not re-learned, not skipped. Get that wrong and the model answers confidently and wrongly, with nothing crashing to tell you. It is called *training/serving skew*, and it is the most expensive bug in this whole course.

**STAGE 8****Monitoring**

Models decay, because the world moves. A new scanner, a units change upstream, a different patient population — and the incoming data stops looking like the training data. That shows up in the *distribution of your predictions* long before anyone reports a wrong diagnosis, which is why you watch it. When you detect it, you collect fresh ground truth and decide whether to retrain — and that decision sends you all the way back to Stage 1. That is the red dashed arrow in the life-cycle diagram, and it is why the diagram is a circle.

---

**5 Wrap-up***≈ 15 minutes***5.1 Exercise: a second model, and a decision** (60 %)

A `DecisionTreeClassifier` works completely differently: instead of weighting features, it asks a series of yes/no questions. `max_depth=3` allows it only three questions deep, which makes it simple enough to draw on a whiteboard for a clinician. In `TODD 1`:

1. Train one on `X_train_scaled`, `y_train`.
2. Predict on `X_test_scaled`.
3. Print its accuracy, recall and precision, next to the logistic regression's.
4. In two or three sentences: **which one would you deploy, and why?** Think about recall, about the four missed cases, and about what you would have to explain to the pathology lab.

The marks are in step 4. Steps 1–3 are four lines you can adapt from §3.

**5.2 Reflection: explain it to the clinician** (40 %)

The pathology lab has no machine learning background. In plain language — no formulas, no jargon — answer three questions about the model you just built, in `TODD 2`. Imagine you are writing three sentences in an email, not an exam answer.

| Question             | What to write                                                 |
|----------------------|---------------------------------------------------------------|
| <b>what_it_does</b>  | What does the tool do, in one sentence a doctor would accept? |
| <b>how_good</b>      | How well does it work? Quote a number and say what it means.  |
| <b>what_it_costs</b> | What does it get wrong, and what happens when it does?        |

If you catch yourself writing "recall", "precision" or "the model", rewrite the sentence. Being able to say it without the vocabulary is the point of the exercise.

### 5.3 Before you submit

1. Runtime → Restart and run all — the whole notebook must run top to bottom without errors.
2. File → Download → .ipynb, rename it MEAI502\_Lab2\_<YourName>.ipynb.
3. Upload it to Blackboard.

### 5.4 Quick self-check

1. Why is it wrong to call `scaler.fit(X)` on the whole dataset before splitting?
2. After scaling, the test set's column means are *not* exactly 0. Why is that the correct outcome rather than a mistake?
3. The baseline scored 0.629 accuracy and 0.000 recall. Explain, in one sentence, how both of those numbers can describe the same model.

#### ANSWERS

1. The mean and standard deviation it learns would be computed from the test rows too, so information from the test set reaches the training data. Your reported score becomes optimistic and no longer estimates unseen performance. Split first, fit the scaler on the training set only.
2. Because the test rows were transformed with numbers learned from the *training* rows, not from themselves — which is exactly what happens in production. Means of exactly 0 would mean the scaler had seen the test set.
3. It always answers "benign": it is right about every benign case (63 % of the test set) and wrong about every malignant one, so accuracy is high and recall on malignant is zero.

### 5.5 Next lab: NumPy

You have now used the whole stack without ever looking underneath it. Lab 3 goes one layer down, to the library everything else is built on: creating arrays (`dtype`, `shape`, `ndim`); indexing, slicing and boolean masks; **vectorisation** — replacing loops with expressions, and the speed difference; **broadcasting** — the rule that lets a (3,1) array meet a (1,4) array; aggregations along axes, reshaping and stacking; and why a pandas DataFrame is really a set of NumPy arrays wearing labels.

**Before Lab 3:** skim the NumPy absolute beginner's guide — [numpy.org/doc/stable/user/absolute\\_beginners.html](https://numpy.org/doc/stable/user/absolute_beginners.html)

### 5.6 References

1. Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow* (3rd ed.), Ch. 3.

- 
2. scikit-learn user guide — [scikit-learn.org/stable/user\\_guide.html](https://scikit-learn.org/stable/user_guide.html)
  3. scikit-learn — *Metrics and scoring* — [scikit-learn.org/stable/modules/model\\_evaluation.html](https://scikit-learn.org/stable/modules/model_evaluation.html)
  4. Dua, D. & Graff, C. — UCI Machine Learning Repository, *Breast Cancer Wisconsin (Diagnostic)*.
- 

*MEAI 502 Applied Machine Learning · Lab 2 · Dr. Naila Marir  
College of Engineering, Effat University · Jeddah · Fall 2026*